

Sentiment Analysis

Submitted by,

Name	Roll No.	Registration No.
Nidhi Srivastav	11771024038	241170510038 OF 2024-2025
Paulami Banerjee	11771024040	241170510040 OF 2024-2025
Tanmoy Ghosh	11771024075	241170510075 OF 2024-2025

Under the guidance of

Dr. Manas Ghosh

Dept. of Computer Applications

RCC Institute of Information Technology

PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF COMPUTER APPLICATIONS
RCC INSTITUTE OF INFORMATION TECHNOLOGY



DEPARTMENT OF COMPUTER APPLICATIONS
RCC INSTITUTE OF INFORMATION TECHNOLOGY
[Affiliated to Maulana Abul Kalam Azad University of Technology]
CANAL SOUTH ROAD, BELIAGHATA, KOLKATA-700015

DEPARTMENT OF COMPUTER APPLICATIONS
RCC INSTITUTE OF INFORMATION TECHNOLOGY



TO WHOM IT MAY CONCERN

I hereby recommend that the Project entitled
“Sentiment Analysis” prepared under my supervision by **Nidhi Srivastav (241170510038 OF 2024-2025)**,
Paulami Banerjee (241170510040 OF 2024-2025), and **Tanmoy Ghosh (241170510075 OF 2024-2025)** of
MCA (3rd Semester), may be accepted in partial fulfillment for the degree of **Master of Computer
Applications** under Maulana Abul Kalam Azad University of Technology (MAKAUT).

.....
Project Supervisor
Department of Computer Applications
RCC Institute of Information Technology

Countersigned:

.....
Head
Department of Computer Applications.,
RCC Institute of Information Technology
Kolkata – 700015



CERTIFICATE OF APPROVAL

The foregoing Project is hereby accepted as a credible study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as a prerequisite to the degree for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the project only for the purpose for which it is submitted.

FINAL EXAMINATION FOR
EVALUATION OF PROJECT

1. _____

2. _____

3. _____

(Signature of Examiners)

ACKNOWLEDGEMENT

We acknowledge our overwhelming gratitude & immense respect to our revered guide, Dr. Manas Ghosh (Head of Department, MCA, RCC Institute of Information Technology) under whose scholarly guideline, constant encouragement & untiring patience; we have proud privilege to accomplish this entire project work. We feel enriched with the knowledge & sense of responsible approach we inherited from our guide & shall remain a treasure in our life.

Name: Nidhi Srivastav

Roll No: 11771024038

Name: Paulami Banerjee

Roll No: 11771024040

Name: Tanmoy Ghosh

Roll No: 11771024075

ABSTRACT

Emotions are a representation of how a person is feeling. In this project our goal is to study ResNet50's accomplishment in accurately detecting facial emotions from images and to build a simpler model with ResNet like architecture and compare its accuracy with ResNet50.

In recent years deep learning architectures have displayed state-of-art performance in diverse computer vision tasks including facial emotion recognition. In this project, we analyse the accomplishments of a very efficient convolutional neural network (CNN) model, ResNet50 and a custom lightweight CNN model with a ResNet-like architecture.

CONTENTS

CERTIFICATE OF APPROVAL	I
ACKNOWLEDGEMENT	II
ABSTRACT	III
CONTENTS	IV
LIST OF ABBREVIATIONS.....	V
LIST OF FIGURES	VI
LIST OF TABLES	VII

CHAPTER-1 **Page No**

1.1 INTRODUCTION.....	1
1.2 LITERATURE REVIEW.....	2

CHAPTER-2

2.1 WORK FLOW.....	3-4
2.2 METHODOLOGIES OF IMPLEMENTATION.....	5-7
2.3 SOFTWARE & HARDWARE REQUIREMENTS.....	8
2.4 PROPOSED WORK.....	9

CHAPTER-3

3.1 TEST CASES & SYSTEM VALIDATION.....	10
3.2 PERFORMANCE ANALYSIS.....	11-12

CHAPTER-4

4.1 CONCLUSION.....	13
4.2 CHALLENGES.....	14
4.3 FUTURE SCOPE.....	15
REFERENCES.....	16
APPENDIX.....	17-21

LIST OF ABBREVIATIONS

Abbreviation	Full Form / Meaning
CIFAR	Canadian Institute for Advanced Research
CNN	Convolutional Neural Network
EEG	Electroencephalography
Fer2013	Facial Expression Recognition 2013 Dataset
ReLU	Rectified Linear Unit
ResNet	Residual Network

LIST OF FIGURES

Fig. No.	Description	Page No.
2.1	Sentiment analysis workflow	3
2.1.1	Sample images from FER2013 dataset	3
2.2.4 a)	Convolutional layer	5
2.2.4 b)	Max pooling in CNN	5
2.2.4 c)	Training and test error comparison of 20-layer and 56-layer plain networks on CIFAR-10 dataset	6
2.2.4 d)	Representation of residual block in ResNet	6
2.2.4 e)	Architecture of ResNet50	6
2.2.4 f)	Architecture of custom CNN	7
3.2.1	Test accuracy of a) ResNet50 and b) custom CNN	11
3.2.2	Test loss of a) ResNet50 and b) custom CNN	11
3.2.3	Sample confusion matrices of a) ResNet50 and b) custom CNN for 70-15-15 data split	12

LIST OF TABLES

Table No.	Description	Page No.
1.1	Literature Review	2
3.2	Average test accuracy and average test loss for ResNet50 and custom CNN	11

CHAPTER 1

1.1 Introduction

Facial emotions are a universal form of communication used by humans and play an important role in assessing human behavior. Although recognizing human emotions from frontal positions in illuminated conditions is quite simple and can have an accuracy of up to 98.9%, doing the same in neutral conditions is still quite challenging due to head position, illumination and obstructions.[1][2][3]

In recent years with rise in deep learning algorithm, specifically Convolutional Neural Networks, it has been possible to train models to predict human emotions from facial images even better from humans themselves. This has led to a breakthrough in the field of robotics, medical treatment, customer satisfaction in hotels and restaurants, driver fatigue surveillance, and many other human-computer interaction systems.[1][3]

In this project our goal was to build a lightweight custom CNN model with residual network architecture and study its ability to detect facial emotions. Moreover, the model was compared to ResNet50 to assess its efficiency. We took several approaches from different publications like class weights, data augmentation and callbacks to improve accuracy of our model.

1.2 Literature Review

Table 1.1: Literature Review

SI No	Author Name	Paper Name	Contribution
1	S. Li, W. Deng	Deep facial expression recognition: A survey, 2018	Reviewed various deep learning approaches for recognizing human emotions from facial images.
2	Pramerdorfer, C., Kampel, M.	Facial expression recognition using CNNs: state of the art, 2016	Summarized how CNNs are used for facial expression recognition and highlighted the top-performing methods.
3	Khanzada, A., Bai, C., Celepcikay, F.T.	Facial expression recognition with deep learning: improving on the state of the art, 2020	Improved CNN-based FER methods and demonstrated how they can work in real-world scenarios.
4	I. J. Goodfellow et al.	Challenges in representation learning: Three machine learning contests, 2013	Introduced the FER2013 dataset and discussed common challenges in training FER models.
5	Y. Tang	Deep Learning using Support Vector Machines, 2013	Proposed using L2-SVM loss function with CNNs to enhance FER model performance.
6	Kaggle Dataset	FER2013 Dataset	Provided a standard dataset of facial images with labeled emotions for training and testing models.
7	Nitish Kundu	Exploring ResNet50: Architecture and Implementation	Explained the structure of ResNet50 and how it can be implemented for image classification tasks.
8	Roboflow Blog	What is ResNet-50?	Gave a clear overview of ResNet50's architecture, including skip connections and bottleneck blocks.
9	He, K., Zhang, X., Ren, S., Sun, J.	Deep residual learning for image recognition, 2015	Introduced the ResNet architecture, showing how residual blocks help train very deep networks effectively.
10	Białek, C., Matiolański, A., Grega, M.	Efficient approach to face emotion recognition with CNNs, 2023	Proposed a more efficient CNN-based method for FER and discussed improvements over older models.

CHAPTER 2

2.1 Work Flow

The suggested Sentiment Analysis system's general process, which is based on facial emotion identification, is demonstrated in Fig 2.1. From dataset collection to sentiment prediction, model evaluation, and performance comparison, the procedure outlines the entire pipeline.

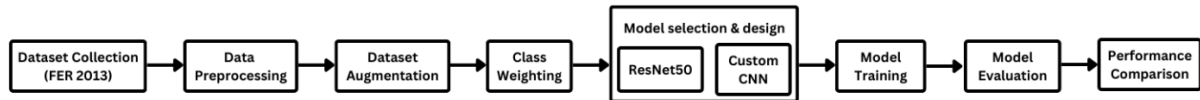


Fig. 2.1: Sentiment analysis workflow

The workflow consists of the following steps:

2.1.1 Dataset Collection:

There are multiple datasets available for FER. We used FER2013 for our study. FER2013 is a widely used standard for emotion detection models and is easily available on Kaggle.[6]

It contains 35,887 grayscale images of 48×48 pixels each and belonging to one of the seven classes of emotions, namely, angry (4,953 images), disgust (547 images), fear (5,121 images), happy (8,989 images), neutral (6,198 images), sad (6,077 images), and surprise (4,002 images).



Fig. 2.1.1: Sample images from FER2013 dataset

2.1.2 Data Preprocessing:

The images were organized into class-specific folders and images were normalized to make model learning more efficient. The images were resized to 224×224×3 for ResNet50 and 48×48×1 for custom CNN. The dataset was also split into three different train-validation-test configurations with image ratios of 60–20–20, 70–15–15, and 80–10–10, respectively

2.1.3 Data Augmentation:

During training, data augmentation was applied using Keras' *ImageDataGenerator*. Images were preprocessed using the model-specific preprocess_input function and augmented with random horizontal flips, translations (up to 10% of image width and height), zooming ($\pm 10\%$), shearing (up to 0.2), and brightness variation (0.8–1.2). Empty pixels introduced by transformations were filled using nearest-neighbour interpolation.

2.1.4 Class Weighting:

To alleviate the class imbalance problem, class weighting was applied which was inversely proportional to the number of samples belong to the respective classes.

2.1.5 Model Selection and Design:

In this project we assessed the accuracy of ResNet50 in correctly predicting the facial emotion of an image shown. Moreover, we used the residual network architecture of used in ResNet50 to design a lightweight custom CNN that can operate on images of size 48×48×1. Henceforth, the accuracy of both the models were studied and compared.

2.1.6 Model Training:

2.1.6.1 ResNet50

ResNet50 is defined with 175 layers in Keras. In this study, the training with ResNet was conducted in two steps. At first all the layers were frozen and the classifier head was trained. This helped the model extract generic features like edges, textures and object parts and to understand the mapping between ImageNet features and our dataset classes. This was trained for 10 epochs. After this the weights were adjusted to the one with the least validation loss.

After this training, the last 50 layers of the ResNet were unfrozen and the model was fine-tuned and trained for 100 epochs. This helped the model capture facial features of the image in more detail.

2.1.6.2 Custom CNN

The custom CNN was trained from scratch using the same dataset and preprocessing pipeline as ResNet50. Training was performed for 100 epochs using the Adam optimizer and all the callbacks used for ResNet50, and the model with the lowest validation loss was selected.

2.1.7 Model Evaluation:

Both the models were trained on 60-20-20 (S1), 70-15-15 (S2) and 80-10-10 (S3) train-test-validation splits for multiple times to infer the stability of the models. To understand the models' efficiency the models were evaluated against the test dataset that was extracted from the FER2013 dataset before the training begun.

2.1.8 Performance Comparison:

In the final step, we evaluated the ResNet50 and custom CNN models' performance for various dataset split ratios. We examined model stability and generalization capacity through this comparison and analyzed parameters such as accuracy trends, loss values and prediction consistency. Based on this analysis, we identified the model that performed more reliably for sentiment analysis using facial emotion recognition.

2.2 Methodologies of Implementation

2.2.1 Convolutional Layer

This is the most characteristic feature of a CNN. In this layer, a filter of a user defined size say 2×2 is glided over the input based on the given stride and the matrix multiplication between the patch of input of dimension equal to the size of the filter and the filter itself is computed. This technique is used to extract features (edges, textures, objects etc.) from the input data.

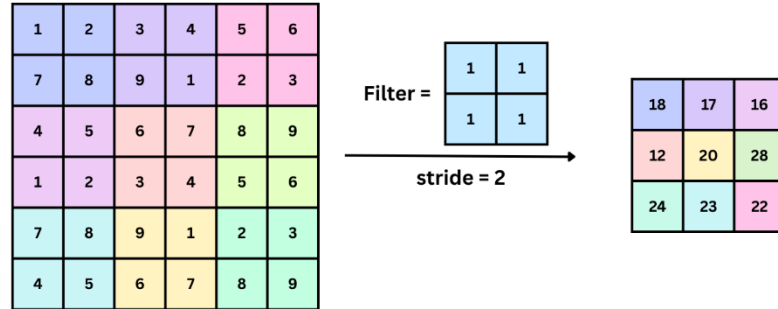


Fig. 2.2.4 a): Convolutional layer

2.2.2 ReLU

ReLU stands for Rectified Linear Unit. It is a popularly used activation function that is used to add non linearity to the dataset. It basically takes the larger value between 0 and the input.

2.2.3 Batch Normalisation

Batch normalisation is a technique used in neural networks to standardize the output of convolutional or linear operations to have 0 mean and unit variance. The standardized data is then scaled and shifted using learnable parameters so the network does not lose expressive power. This technique helps in avoiding vanishing and exploding gradients and during backpropagation.

2.2.4 Pooling

It is a dimensionality reduction technique that helps in downsampling while retaining the most important features. This helps network become more computationally efficient and reduces overfitting by reducing number of parameters.

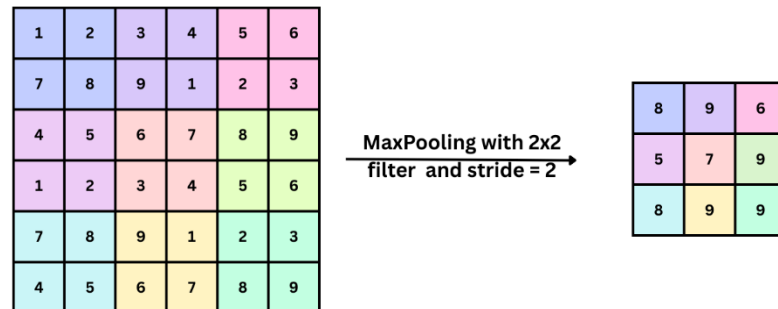


Fig. 2.2.4 b): Max pooling in CNN

To perform a comparison analysis, we deployed two distinct deep learning models.

2.2.5 ResNet50 Model

ResNet50 is a deep convolutional neural network (CNN) architecture that was developed by Microsoft Research in 2015. It is a variant of ResNet architecture which stands for Residual Network and contains 50 layers.[7]

The ResNet architecture has two significant characteristics that makes it robust and helps achieve state-of-the-art results. These characteristics are skip connections and bottleneck blocks.

Bottleneck block: This block in ResNet usually consists of three convolutional layers where each layer is followed by batch normalization and ReLU activation function:

- The first layer uses a filter of size 1×1 and reduces the number of channels of the input. This dimensionality reduction helps to compress the data and improve computational efficiency without sacrificing too much information.
- The second convolutional layer might use a filter size of 3×3 to extract spatial features from the data.
- The third convolutional layer again uses a filter size of 1×1 to restore the original number of channels before the output is added to the shortcut connection.[8]

Skip connections: As networks become deeper, their accuracy saturates and then degrades rapidly. This is not due to overfitting or underfitting but due to difficulty of optimising the training process

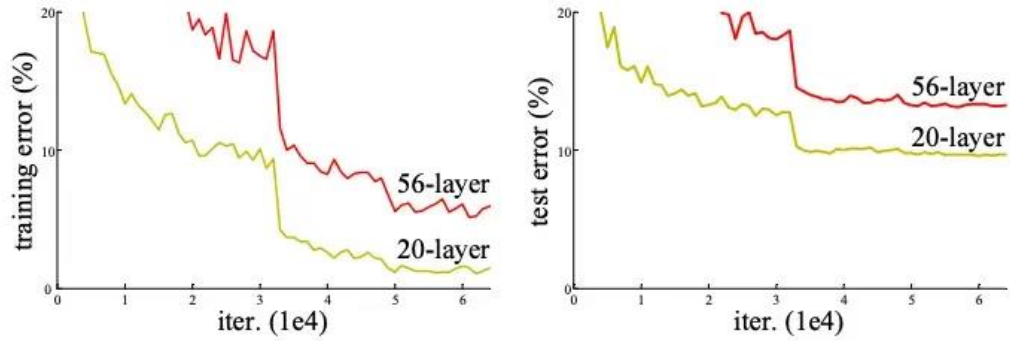


Fig.2.2.4 c): Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error.[9]

ResNet solved this problem using Residual Blocks that allow for the direct flow of information through the skip connections, solving the vanishing gradient problem.

The bottleneck block is used as the residual block in ResNet. The structure is as shown:

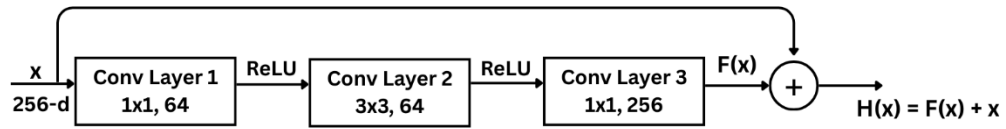


Fig. 2.2.4 d): Representation of residual block in ResNet

So, using bottleneck blocks and skip connections, ResNet helps deep networks computationally feasible and scalable to large datasets like ImageNet.

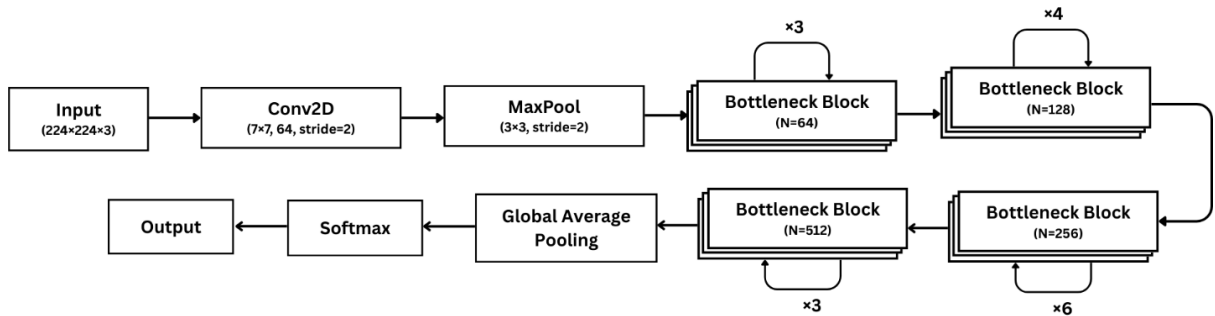


Fig. 2.2.4 e): Architecture of ResNet50

2.2.6 Custom CNN

The custom CNN model was designed with a similar architecture as ResNet50. This model contains residual blocks and skip connections to mimic ResNet architecture but contains much fewer layers.

In this model, only 6 bottleneck blocks were used compared to 16 in ResNet50. Moreover, this model took inputs of size $48 \times 48 \times 1$ compared to $224 \times 224 \times 3$ in ResNet50.

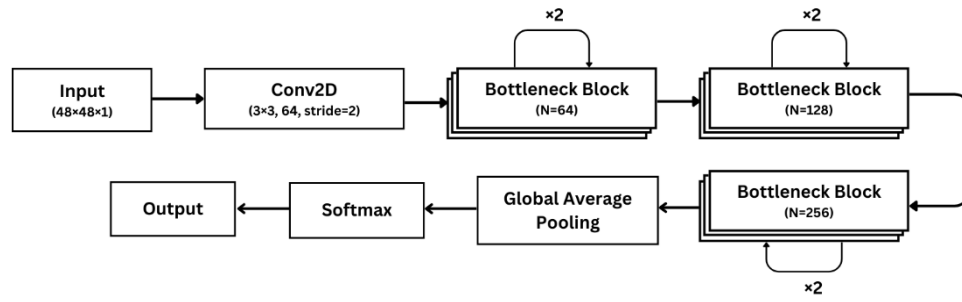


Fig. 2.2.4 f): Architecture of custom CNN

Overall, this custom CNN model is a much lighter version of the deep neural network ResNet50.

2.3 Software & Hardware Requirements

2.3.1 Software Specifications

This project is a computationally heavy project and requires the use of GPU for fast and efficient execution. For this, cloud-based platforms namely, Kaggle Notebook and Google Colab were used which were available free-of-cost but came with limited time usage.

Python was used as the language for study because of availability of robust libraries for applications involving image processing and deep learning and its compact syntax. For implementing and training the facial emotion recognition models, we used deep learning frameworks such as TensorFlow and Keras. Additional libraries like NumPy and Pandas were used for data handling and preprocessing, while Matplotlib helped us visualize training results such as accuracy and loss curves. For performance evaluation and analysis, Scikit-learn was used to generate evaluation metrics and reports. Overall, this software setup provided a flexible and efficient environment for developing and testing our models

Software Requirements

- **Platform:** Google Colab, Kaggle Notebook
- **Programming Language:** Python
- **Deep Learning Frameworks:** TensorFlow, Keras
- **Libraries:** NumPy, Pandas, Matplotlib, Scikit-learn
- **Web Browser:** Google Chrome or any modern web browser

2.3.2 Hardware Specifications

The project did not require any expensive local hardware.

Hardware Requirements:

- **Local System:** Any standard laptop or desktop
- **Processor:** Basic CPU (for accessing cloud platforms)
- **RAM:** Minimum 4 GB
- **Internet Connection:** Stable broadband connection
- **Cloud Resources:** CPU/GPU provided by Kaggle and Google Colab

2.4 Proposed Work

Facial emotion recognition is a widely used feature and ResNet with his unique architecture has provided state-of-art accuracy in accurately predicting emotions from images. But it faces two challenges — it is a heavyweight model and it requires images with three channels.

Our custom CNN was designed with residual network architecture to work with much fewer layers and grayscale images as input.

The proposed work mainly focuses on:

- Using transfer learning and fine-tuning to alter an already-trained model (ResNet50)
- Designing a new lightweight CNN architecture with residual connections
- Using class weighting to address class imbalance
- Improving generalization through data augmentation
- Making a thorough analysis of the two models' performances
- Our goal in making these changes and design decisions was to create a face expression recognition-based sentiment analysis system that is lightweight but at par in performance with heavyweight model like ResNet50

CHAPTER 3

3.1 Test Cases & System Validation

The training was carried out for multiple train-test-validation splits, 60-20-20, 70-15-15 and 80-10-10 where the images were chosen randomly and each split was studied multiple times.

In each split, the model was trained on training dataset and validated using the validation dataset. The test dataset was kept in a separate location during training. After the training the model performance was assessed based on its ability to correctly predicting the facial emotions of test images.

Similar to training and validation dataset, the test dataset was also augmented with same measures to better understand the model's accuracy under non-ideal conditions.

To analyse the models' performance, test accuracy, test loss and confusion matrices were recorded.

3.2 Performance Analysis

3.2.1 Metrics

- Training loss: The value of the loss function that represents the model’s error on the data it is learning from.
- Training accuracy: Model’s ability in predicting the correct class for the data it is being trained on.

3.2.2 Analysis

Both the models were trained on 60-20-20 (S1), 70-15-15 (S2) and 80-10-10 (S3) train-test-validation splits for multiple times to infer the stability of the models. To understand the models’ efficiency the models were evaluated against the test dataset that was extracted from the FER2013 dataset before the training begun.

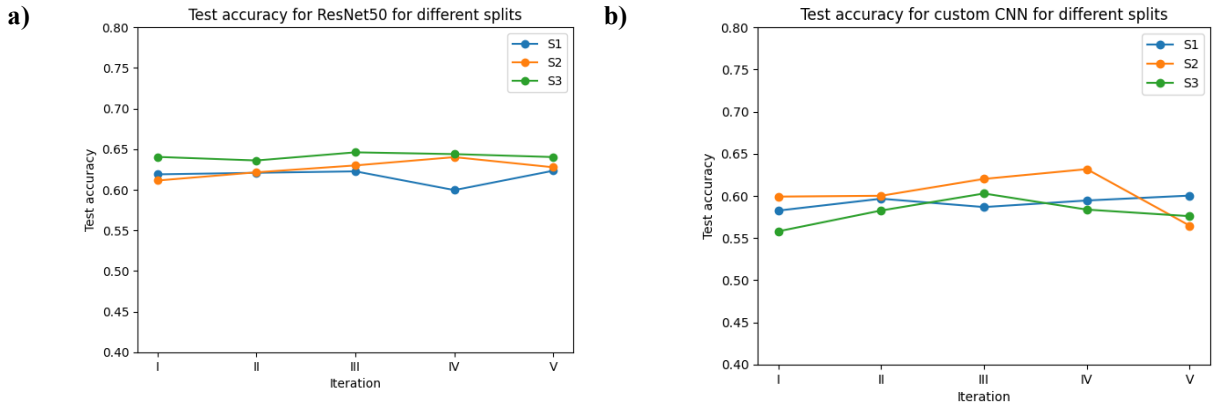


Fig. 3.2.1: Test accuracy for a) ResNet50 and, b) custom CNN

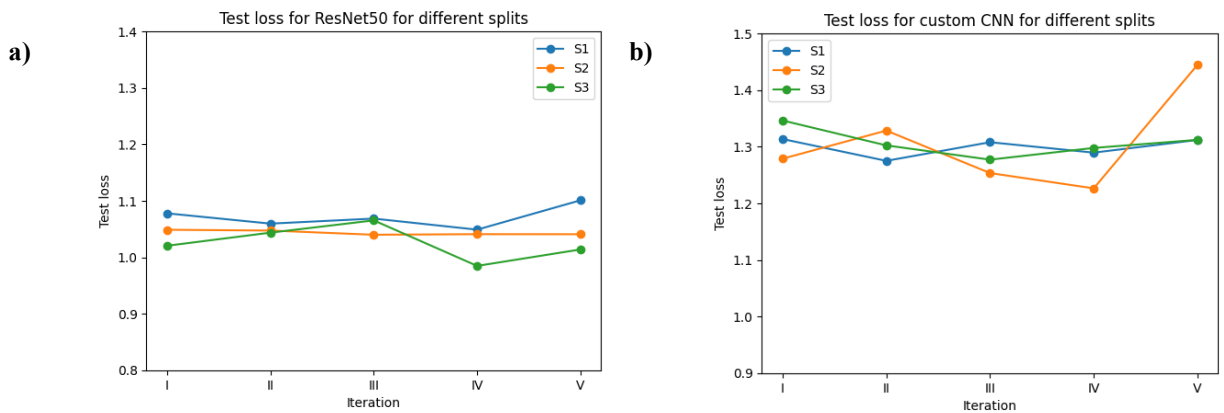


Fig. 3.2.2: Test loss for a) ResNet50 and, b) custom CNN

Table 3.1: Average test accuracy and average test loss for ResNet50 and custom CNN

Model	Split	Average Test Accuracy	Average Test Loss
ResNet50	60-20-20	61.70%	1.07114
	70-15-15	62.60%	1.04352
	80-10-10	64.12%	1.02556
Custom CNN	60-20-20	59.21%	1.29956
	70-15-15	60.31%	1.30634
	80-10-10	58.05%	1.30712

To understand the class-wise accuracy for ResNet50 and custom CNN, the confusion matrix was plotted. From the confusion matrix, Fig. 3.2.3.a, it can be seen that ResNet50 performed exceptionally well for classes ‘happy’ and

‘surprise’ and satisfactorily for classes ‘angry’, ‘disgust’, ‘neutral’, however, the model performed poorly for ‘fear’ and ‘sad’ classes—common difficulties in FER2013-based classification.

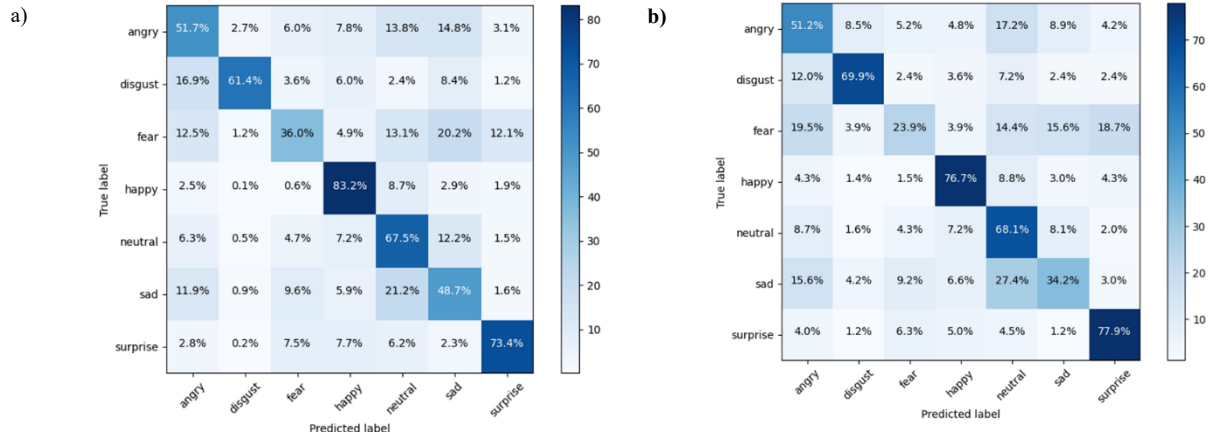


Fig. 3.2.3: Sample confusion matrices for **a)** ResNet50 and **b)** custom CNN for 70-15-15 split

The confusion matrix, **Fig. 3.2.3.b**, it can be seen that custom CNN is at par with ResNet50 for predicting human emotions. Although custom CNN has poorer performance for ‘sad’ and ‘fear’ classes even compared to ResNet50 but it outperforms ResNet50 for classes ‘disgust’, ‘neutral’ and ‘surprise’.

CHAPTER 4

4.1 Conclusion

For this study, the FER2013 dataset was used which is the benchmark for facial emotion recognition. The dataset came with a major challenge which is huge class imbalance. This was alleviated by employing class weights and data augmentation methods. Our custom CNN model achieved an average accuracy of 59.19% accuracy compared to the 62.80% accuracy of ResNet50. So, our model's accuracy is much lower than ResNet-50 and also human evaluators who have achieved 65% of accuracy in the same dataset [10].

The main limitation of our study is that we trained our model only on FER2013 whereas most of the models with better accuracy are trained on multiple datasets. Although the custom CNN achieves a slightly lower overall accuracy than ResNet50, its class-wise performance is comparable to—and in some cases exceeds—that of ResNet50 for certain emotions.

4.2 Challenges

We ran across a number of practical and technological challenges when implementing the face emotion-based sentiment analysis system into use. These difficulties mostly resulted from computational constraints, model complexity, and data restrictions.

1. Class Imbalance in Dataset

The dataset used, FER2013, is a hugely imbalanced dataset with as many as 8,989 images for ‘happy’ class and as low as 547 images in ‘disgust’ class. This imbalance was handled by adding class weights proportional to inverse of no. of images in each class to images belonging to each class.

2. Overfitting on Training Data

There were signs of overfitting in the dataset which were curbed using features like dropout and data augmentation.

3. Low Resolution and Grayscale Images

There are low-resolution grayscale (48 x 48 pixel) photos in the FER2013 collection. It was difficult to extract fine-grained emotional traits because of the inadequate face information, especially for related emotions like surprise and terror.

4. Computational Limitations

Since the experiments were conducted on Google Colab and Kaggle Notebooks, we faced limitations related to GPU availability, session timeouts, and memory constraints. These limitations restricted batch size selection and prolonged training time for deeper models like ResNet50.

5. Confusion Between Similar Emotions

The overlap between several emotion types presented another difficulty. Because of their comparable facial features, emotions like ‘sad’ and ‘neutral’ or ‘fear’ and ‘surprise’ can occasionally be misclassified.

6. Hyperparameter Tuning

Selecting optimal hyperparameters such as learning rate, batch size, and number of epochs required multiple experiments. This process was time-consuming and required balancing performance with computational efficiency.

4.3 Future Scope

To further improve the accuracy, we would further train our model on multiple different datasets and would also combine other factors like EEG data to back our model predictions more accurately. Furthermore, we plan to apply this in real world by developing a mobile and web application with real-time recognition speeds.

References

- [1] S. Li and W. Deng, “Deep facial expression recognition: A survey,” arXiv preprint arXiv:1804.08348, 2018.
- [2] Pramerdorfer, C., Kampel, M.: Facial expression recognition using convolutional neural networks: state of the art. Preprint arXiv:1612.02903v1, 2016.
- [3] Khanzada, A., Bai, C., Celepcikay, F.T.: Facial expression recognition with deep learning: improving on the state of the art and applying to the real world. Preprint arXiv:2004.11823v1, 2020.
- [4] I. J. Goodfellow, D. Erhan, P. L. Carrier, A. Courville, M. Mirza, B. Hamner, W. Cukierski, Y. Tang, D. Thaler, D.-H. Lee et al., “Challenges in representation learning: A report on three machine learning contests,” in International Conference on Neural Information Processing. Springer, 2013, pp. 117–124.
- [5] Y. Tang, “Deep Learning using Support Vector Machines,” in International Conference on Machine Learning (ICML) Workshops, 2013.
- [6] <https://www.kaggle.com/datasets/msambare/fer2013>
- [7] <https://medium.com/@nitishkundu1993/exploring-resnet50-an-in-depth-look-at-the-model-architecture-and-code-implementation-d8d8fa67e46f>
- [8] <https://blog.roboflow.com/what-is-resnet-50/>
- [9] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. Preprint arXiv:1512.03385, 2015.
- [10] Białek, C., Mاتیolański, A., Grega, M.: An efficient approach to face emotion recognition with convolutional neural networks. Electronics 12(12), 2707, 2023.

Appendix

ResNet50

Model Training

```
base_model = tf.keras.applications.ResNet50(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)
for layer in base_model.layers:
    layer.trainable = False
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dropout(0.3)(x)
output = Dense(
    7,
    activation='softmax'
)(x)
model_1 = Model(
    inputs=base_model.input,
    outputs=output
)
model_1.compile(
    loss='categorical_crossentropy',
    optimizer=Adam(0.001),
    metrics=['accuracy']
)
```

Finetuning

```
for layer in base_model.layers[-50:]:
    layer.trainable = True
model_1.compile(
    loss="categorical_crossentropy",
    optimizer=Adam(1e-5),
    metrics=['accuracy']
)
```

Custom CNN

Bottleneck Block

```
def bottleneck_block(x, filters, strides=1):
    shortcut = x
    y = layers.Conv2D(
        filters,
        (1, 1),
        strides=1,
        padding='same'
    )(x)
    y = layers.BatchNormalization()(y)
    y = layers.ReLU()(y)
    y = layers.Conv2D(
        filters,
        (3, 3),
        strides=strides,
        padding='same'
    )(y)
    y = layers.BatchNormalization()(y)
    y = layers.ReLU()(y)
    y = layers.Conv2D(
        filters * 4,
        (1, 1),
        strides=1,
        padding='same'
    )(y)
    y = layers.BatchNormalization()(y)
    if strides != 1 or x.shape[-1] != filters * 4:
        shortcut = layers.Conv2D(
            filters * 4,
            (1, 1),
            strides=strides,
            padding='same'
        )(shortcut)
        shortcut = layers.BatchNormalization()(shortcut)
    output = layers.add([y, shortcut])
    output = layers.ReLU()(output)
    return output
```

Model Designing

```
def build_advanced_custom_model(input_shape, num_classes):
    inputs = tf.keras.Input(shape=input_shape)
    x = layers.Conv2D(64, (3, 3), padding='same')(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    x = bottleneck_block(x, 64)
    x = bottleneck_block(x, 64)
    x = bottleneck_block(x, 128, strides=2)
    x = bottleneck_block(x, 128)
    x = bottleneck_block(x, 256, strides=2)
    x = bottleneck_block(x, 256)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(
        num_classes,
        activation='softmax'
    )(x)
    model = Model(
        inputs=inputs,
        outputs=outputs
    )
    loss_fn = tf.keras.losses.CategoricalCrossentropy(
        label_smoothing=0.1
    )
    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=0.001
        ),
        loss=loss_fn,
        metrics=['accuracy']
    )
    return model
```

Model Building

```
model_2=build_advanced_custom_model(input_shape=(48, 48, 1), num_classes=7)
```

Callbacks

Checkpoint

```
checkpoint = ModelCheckpoint(
    os.path.join(WORKING_DIR, "model.keras"),
    monitor="val_accuracy",
    save_best_only=True,
    mode="max",
    verbose=1
)
```

Early Stopping

```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=8,
    restore_best_weights=True,
    verbose=1
)
```

Reduce Learning Rate

```
reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.2,
    patience=3,
    min_lr=1e-8,
    verbose=1
)
```

Confusion Matrix

```
cm = confusion_matrix(y_true, y_pred)
cm_percent = (
    cm.astype(float)
    / cm.sum(axis=1)[:, np.newaxis]
    * 100
)
plt.figure(figsize=(8, 6))
plt.imshow(
    cm_percent,
    interpolation='nearest',
    cmap=plt.cm.Blues
)
plt.title('Confusion Matrix For ResNet50 (Percentage)')
plt.colorbar()
tick_marks = np.arange(len(LABELS))
```

```

plt.xticks(tick_marks, LABELS, rotation=45)
plt.yticks(tick_marks, LABELS)
fmt = '.1f' # show one decimal place
thresh = cm_percent.max() / 2.0
for i, j in itertools.product(
    range(cm_percent.shape[0]),
    range(cm_percent.shape[1])
):
    plt.text(
        j,
        i,
        format(cm_percent[i, j], fmt) + '%',
        horizontalalignment='center',
        color='white' if cm_percent[i, j] > thresh else 'black'
    )
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.tight_layout()
plt.show()

```